OPEN ROBOT OPERATION COMMANDS

OpenROC

The Language of Physical AI

Protocol Specification

OpenROC Community

October 13, 2026

Protocol **OpenROC** openroc-beta-1.0

Encodings: OpenROC-JSON • OpenROC-BIN

Contents

1 Introduction to OpenROC **2**

1.1 What is OpenROC? . . . 2
1.2 Design Goals . . . 2
1.3 The OpenROC Stack . . . 2
1.4 The Two Operation Stacks . . . 4
1.5 The Command Model . . . 4
1.6 Encodings . . . 5
1.7 Adapters . . . 5
1.8 Who OpenROC Is For . . . 5
1.9 Conformance . . . 5
1.10 Summary . . . 6

2 OpenROC Core **6**

2.1 Overview . . . 6
2.2 Message Structure . . . 6
2.2.1 Envelope Fields . . . 6
2.2.2 Addressing . . . 8
2.2.3 Priority and Safety Classes . . . 8
2.2.4 Quality of Service . . . 8
2.3 Command Semantics . . . 9
2.4 Execution Lifecycle . . . 9
2.4.1 Acknowledgement Messages . . . 10
2.5 Correlation and Idempotency . . . 11
2.6 Versioning . . . 11
2.7 Minimal Core Conformance . . . 12

3 OpenROC Action Stack **12**

3.1 Overview . . . 12
3.2 The Action Plane . . . 12
3.3 Objective . . . 12
3.3.1 Goal and Success Criteria . . . 13
3.3.2 Constraints . . . 14
3.3.3 Authority and Confidence . . . 14
3.4 Action Verbs . . . 15
3.5 Action Lifecycle . . . 16
3.6 The Action Loop . . . 16
3.7 Human-in-the-Loop . . . 17
3.8 Policy . . . 17
3.9 Action Feedback . . . 17
3.10 Task and Skill . . . 18

3.10.1 Task Graph . . . 18
3.10.2 Movement and Trajectories . . . 19
3.10.3 Execution Modes . . . 20
3.10.4 Time and Frames . . . 20
3.11 From Action to Actuation . . . 21
3.12 Conformance Notes . . . 21

4 OpenROC Control Stack 21
4.1 Overview . . . 21
4.2 Concepts . . . 21
4.2.1 Actuator . . . 22
4.2.2 Control Modes . . . 22
4.2.3 Setpoint . . . 22
4.3 Message Verbs . . . 23
4.3.1 Set Example . . . 23
4.4 Streaming Control . . . 24
4.5 Timing and Determinism . . . 24
4.6 Feedback . . . 25
4.7 Safety Integration . . . 25
4.8 Pipeline . . . 26
4.9 Conformance Notes . . . 27

5 OpenROC-JSON 27
5.1 Overview . . . 27
5.2 Design Rules . . . 27
5.3 Numbers . . . 27
5.4 Time . . . 28
5.5 Binary Data . . . 29
5.6 Canonical Example . . . 29
5.7 Schemas . . . 29
5.8 Streaming . . . 30
5.9 Interoperability with BIN . . . 31
5.10 Validation Rules . . . 31
5.11 Conformance Notes . . . 31

6 OpenROC-BIN 31
6.1 Overview . . . 31
6.2 Framing . . . 32
6.3 Varints . . . 32
6.4 The Field Table . . . 32
6.5 Strings and Common Values . . . 33
6.6 Payload Encoding . . . 33

6.7 Example . . . 34
6.8 Numeric Encoding . . . 34
6.9 Fragmentation and Compression . . . 35
6.10 Checksums and Integrity . . . 35
6.11 Transport Independence . . . 35
6.12 Conformance Notes . . . 36

7 OpenROC Adapters 36
7.1 Overview . . . 36
7.2 The Adapter Contract . . . 36
7.2.1 Endpoint Descriptor . . . 36
7.3 Capability Discovery . . . 37
7.4 Common Adapters . . . 38
7.5 Translation Rules . . . 38
7.6 Bidirectional Flow . . . 39
7.7 Connection and Reconnection . . . 39
7.8 Addressing and Routing . . . 39
7.9 Security at the Boundary . . . 40
7.10 Conformance Notes . . . 40

8 Security & Safety 40
8.1 Overview . . . 40
8.2 Identity and Authentication . . . 40
8.3 Command Integrity . . . 41
8.4 Replay Protection . . . 41
8.5 Safety Classes in Practice . . . 42
8.6 Interlocks and Envelope Limits . . . 42
8.7 Protective Stop . . . 42
8.8 Watchdogs . . . 43
8.9 Geofencing and Zones . . . 43
8.10 Rate Limiting and Command Budgets . . . 43
8.11 Audit and Traceability . . . 43
8.12 Threat Model Summary . . . 43
8.13 Conformance Notes . . . 45

9 Testing & Validation 45
9.1 Overview . . . 45
9.2 Test Levels . . . 45
9.3 Test Vectors . . . 46
9.4 Conformance Suite . . . 47
9.4.1 Behavioral Tests . . . 47
9.5 Simulation . . . 47

9.6 Hardware-in-the-Loop . . . 48
9.7 Fuzzing and Robustness . . . 48
9.8 Interoperability Matrix . . . 49
9.9 Continuous Integration . . . 49
9.10 Conformance Notes . . . 49

10 Deployment & Updates 49
10.1 Overview . . . 49
10.2 Deployment Topology . . . 50
10.3 Version Negotiation . . . 50
10.4 Capability Handshake . . . 51
10.5 Update Artifacts . . . 51
10.6 Update Flow . . . 52
10.7 Atomicity and Rollback . . . 53
10.8 Staged Rollout . . . 53
10.9 Coordinated Updates . . . 53
10.10 Monitoring After Update . . . 53
10.11 Security of Updates . . . 54
10.12 Conformance Notes . . . 54

1 Introduction to OpenROC

1.1 What is OpenROC?

OpenROC (Open Robot Operation Commands) is an open protocol that connects AI intelligence to robots, actuators, and physical systems through standardized commands. It defines a common language that an AI system can use to express intent, and that machines can use to execute that intent safely and deterministically.

OpenROC is not a robot operating system, a middleware product, or a vendor SDK. It is a **protocol**: a precise specification of message structure, command semantics, and execution lifecycle that any conforming implementation can speak. Because it is transport-agnostic and hardware-neutral, OpenROC lets a decision-making model on a workstation, an onboard controller, and a fleet of actuators share one coherent vocabulary.

> **OpenROC — The Language of Physical AI.**

1.2 Design Goals

OpenROC is organized around a small number of goals that guide every design decision:

1. **Separation of intent from execution.** An intent ("pick up the red box") is expressed independently of the actuation ("apply 4.2 N·m to joint 3"). Each level can evolve, be certified, and be replaced without disturbing the other.
2. **One protocol, two encodings.** Every message has an exact representation in both **OpenROC-JSON** (human-readable) and **OpenROC-BIN** (compact binary). The two encodings are semantically identical; only their wire representation differs.
3. **Safety as a first-class property.** Every command carries an explicit safety class, deadline, and authority. Safety systems can inspect, admit, or reject commands before they reach hardware.
4. **Determinism where it matters.** Real-time and safety-critical paths have bounded latency and predictable framing, while best-effort and telemetry paths remain flexible.
5. **Interoperability by construction.** OpenROC Adapters let existing robots, actuators, and middleware participate without changing their native protocols.

1.3 The OpenROC Stack

OpenROC is composed of peer components, each addressing one concern:

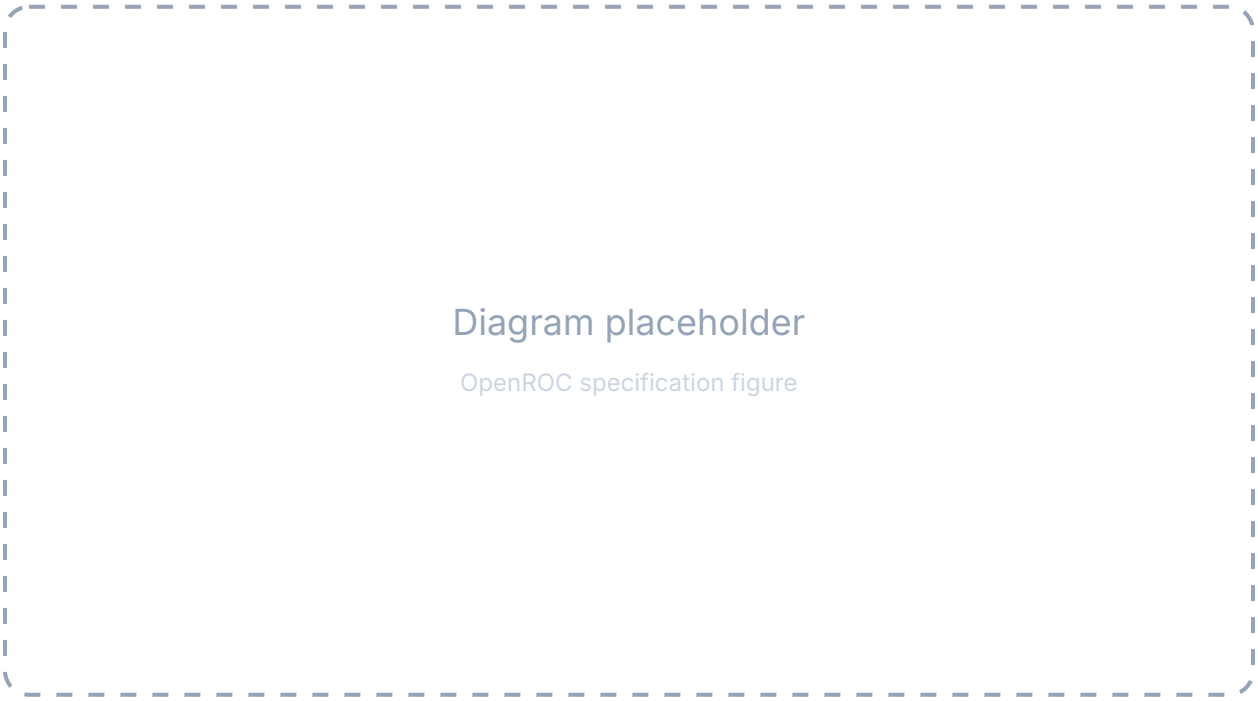


Figure 1: OpenROC Stack

Component	Responsibility
OpenROC Core	Message structure, command semantics, and execution lifecycle
OpenROC Action Stack	AI decisions, objectives, high-level robot tasks, movements, and trajectories
OpenROC Control Stack	Low-level hardware control
OpenROC-JSON	Human-readable encoding
OpenROC-BIN	Compact binary encoding
OpenROC Adapters	Integration with existing robots, actuators, and middleware

Two of these components form the **operation stacks** described in this specification: the **Action Stack** and the **Control Stack**. Core binds them together with a shared envelope and lifecycle. JSON and BIN encode them on the wire, and Adapters connect them to the physical world.

1.4 The Two Operation Stacks

OpenROC models authority as a downward flow from reasoning to motion, organized into two stacks:

- **OpenROC Action Stack** answers *what to do, and why*. It carries objectives, goals, constraints, confidence, and policy, emitted by a planner or model when it concludes that some outcome should be pursued, and it converts a committed objective into concrete tasks, skills, and trajectories — for example a waypoint list for a manipulator or a navigation goal for a mobile base.
- **OpenROC Control Stack** answers *how*. It carries device-level setpoints — position, velocity, torque, impedance, or current — together with control modes and real-time constraints.

Each stack consumes the output of the stack above it and emits feedback upward. An objective may produce zero or more tasks; a task may produce many control commands; the Control Stack always produces telemetry and status.

On the wire, messages identify the plane they belong to (`action` or `actuation`); the Action Stack carries the first, and the Control Stack carries the second.

1.5 The Command Model

Everything in OpenROC is expressed as a **message**. Messages are typed, addressed, and correlated. The Core envelope is shared by all planes:

```
{
  "roc": "1.0",
  "id": "01J8Z9Q7V6B4M2T0R3C5K8P1XA",
  "ts": "2026-10-13T09:00:00.000Z",
  "type": "command",
  "plane": "action",
  "verb": "move",
  "source": "planner.workcell-7",
  "target": "robot.arm-1",
  "priority": "operational",
  "deadline": "2026-10-13T09:00:05.000Z",
  "correlation": "01J8Z9Q7V6B4M2T0R3C5K8P1AB",
  "payload": { "skill": "pick", "object": "red_box" }
}
```

A message flows through a well-defined lifecycle — issued, admitted, validated, scheduled, executing, and completed (or failed, rejected, expired, or cancelled). The lifecycle is described in **OpenROC Core** and is identical across all planes, so tooling, logging, and safety inspection can be written once.

1.6 Encodings

OpenROC defines two normative encodings of the same semantic model:

- **OpenROC-JSON** is the human-readable encoding. It is used for development, debugging, configuration, tooling, and any link where bandwidth is not constrained. It is a strict profile of JSON with documented casing, number, unit, and time rules.
- **OpenROC-BIN** is the compact binary encoding. It uses fixed and variable-length framing, integer opcodes, and checksums to minimize bytes and decode time. It is used on constrained links, high-rate control streams, and safety-critical paths.

Because both encodings are normative, a conforming implementation must be able to convert losslessly between them. Interop tests in the conformance suite verify this property.

1.7 Adapters

Real deployments rarely replace all hardware at once. **OpenROC Adapters** bridge OpenROC to the protocols already in use — ROS 2, CAN, CANopen, EtherCAT, Modbus, MQTT, gRPC, OPC UA, and vendor SDKs. An adapter declares the capabilities it exposes, translates OpenROC messages into native operations, and translates native feedback back into OpenROC events. Adapters make OpenROC an overlay on existing infrastructure rather than a rip-and-replace requirement.

1.8 Who OpenROC Is For

- **AI and robotics researchers** who need a stable interface between learning/planning systems and physical execution.
- **Robot and actuator vendors** who want one integration to reach every OpenROC-capable controller.
- **Integrators** who must combine robots from multiple vendors into a single commanded system.
- **Safety engineers** who need to reason about, constrain, and audit every command that can move a machine.

1.9 Conformance

An implementation is **OpenROC-conformant** when it:

1. Accepts and produces OpenROC Core envelopes with the required fields and lifecycle transitions.
2. Correctly encodes and decodes at least one plane in OpenROC-JSON and OpenROC-BIN.
3. Enforces the safety class, deadline, and authority rules of every command it admits.
4. Passes the OpenROC conformance test suite for its declared capability profile.

Conformance is scoped by **capability profile**, so a small actuator controller need only implement the Control Stack, while a fleet controller may implement both stacks.

1.10 Summary

OpenROC gives physical AI a shared language. The **Action Stack** turns AI objectives into concrete robot tasks, the **Control Stack** turns those tasks into precise device control, two encodings carry the same semantics, safety is intrinsic to every command, and adapters integrate the robots and actuators that already exist. The remaining chapters specify each component in detail.

OpenROC — The Language of Physical AI.

2 OpenROC Core

2.1 Overview

OpenROC Core defines the message structure, command semantics, and execution lifecycle shared by every operation plane. It is the smallest part of OpenROC that a device must implement to participate in the protocol, and it is the contract that makes Action and Actuation messages mutually intelligible.

Core is intentionally domain-neutral. It says nothing about robots, motion, or hardware. Instead it standardizes how a message is addressed, classified, bounded in time, authorized, correlated, and tracked from issue to completion.

2.2 Message Structure

Every OpenROC message is an **envelope** with a plane-specific **payload**. The envelope is fixed by the protocol; the payload is defined by the plane that owns it.

2.2.1 Envelope Fields

Field	Required	Description
`roc`	yes	Protocol version, e.g. `"1.0"`.
`id`	yes	Unique message identifier (ULID/UUIDv7). Used for idempotency and dedup.

Field	Required	Description
`ts`	yes	Issue time as an RFC 3339 timestamp with millisecond precision.
`type`	yes	`command`, `event`, `query`, `response`, `telemetry`, or `ack`.
`plane`	yes	`action` or `actuation`.
`verb`	yes	Plane-defined operation, e.g. `move`, `grasp`, `set_torque`.
`source`	yes	Logical address of the origin.
`target`	yes	Logical address of the intended endpoint.
`priority`	yes	Safety/priority class.
`deadline`	no	Absolute time by which the command must have completed.
`ttl`	no	Relative validity window, as an ISO 8601 duration.
`correlation`	no	`id` of the message this one responds to or belongs to.
`sequence`	no	Monotonic per-(source,target) counter for ordered streams.
`qos`	no	Requested delivery guarantees.
`payload`	yes	Plane-specific body.
`integrity`	no	Digest and/or signature over the canonical message bytes.
`meta`	no	Extensions namespace, ignored if unknown.

2.2.2 Addressing

Addresses are hierarchical dotted paths that name logical endpoints, not network locations:

```
robot.arm-1
robot.arm-1.joint-3
fleet.warehouse-a.agv-12
cell.weld-2.gripper
```

The first segment names an administrative scope; the final segment names a device or resource. Discovery (see **OpenROC Adapters**) maps addresses to concrete transports. A target may be a **wildcard** for broadcast-style events:

```
fleet.warehouse-a.*
robot.arm-1.*
```

2.2.3 Priority and Safety Classes

Priority	Meaning	Typical Use
`emergency`	Stop/halt semantics. Preempts everything.	E-stop, protective stop.
`safety`	Safety-relevant control; must be admitted before execution.	Envelope limits, interlocks.
`operational`	Normal commanded work.	Pick, place, navigate.
`best_effort`	Non-critical, droppable.	Logging, hints.

`emergency` and `safety` messages may not be silently dropped and may not be delayed behind `operational` traffic.

2.2.4 Quality of Service

`qos` expresses transport-independent delivery intent:

```
{
  "delivery": "at_least_once",
  "ordering": "per_source",
  "durability": "volatile",
  "max_latency_ms": 20
```

```
}
```

Adapters map these to native mechanisms (see the Adapters chapter). Core only requires that the requested guarantees are either honored or explicitly downgraded and reported.

2.3 Command Semantics

A **command** is a request to change state. Core defines the semantics that all commands share, regardless of plane:

1. **Idempotency.** Two commands with the same `id` from the same `source` are the same command. Re-delivery must not cause a duplicate effect.
2. **Bounded validity.** A command is valid only within its `ttl` and must complete before its `deadline`. An expired command is rejected and never executed.
3. **Explicit authority.** A command is executable only if its `source` is authorized for its `target`, `plane`, and `verb`.
4. **Preemption.** A higher-priority command may preempt a lower-priority one on the same target. The preempted command transitions to `preempted`.
5. **Atomic admission.** A command is either admitted and then executed, or rejected before any effect. There is no partial admission.
6. **Observable outcome.** Every admitted command eventually produces exactly one terminal event: `completed`, `failed`, `cancelled`, `expired`, or `preempted`.

2.4 Execution Lifecycle

Every OpenROC command moves through a state machine. States are reported with `ack` and `event` messages.

State	Description
`issued`	The command exists at the source and has been transmitted.
`received`	The target has decoded the envelope.
`admitted`	Authority, safety class, deadline, and resource checks passed.
`rejected`	A check failed; the command has no effect.
`validated`	The plane-specific payload was validated against its schema.

State	Description
`scheduled`	Resources were reserved and the command was placed in the execution queue.
`executing`	The command is affecting the system.
`completed`	The command reached its goal.
`failed`	The command could not reach its goal; a fault is reported.
`preempted`	A higher-priority command took over the target.
`cancelled`	The source or an operator withdrew the command before completion.
`expired`	The command's `ttl` or `deadline` elapsed before completion.

2.4.1 Acknowledgement Messages

The target acknowledges lifecycle transitions with `ack` messages that share the command's `id` in `correlation`:

```
{
  "roc": "1.0",
  "id": "01J8Z9Q7V6B4M2T0R3C5K8P1XB",
  "ts": "2026-10-13T09:00:00.012Z",
  "type": "ack",
  "plane": "action",
  "verb": "ack",
  "source": "robot.arm-1",
  "target": "planner.workcell-7",
  "priority": "operational",
  "correlation": "01J8Z9Q7V6B4M2T0R3C5K8P1XA",
  "payload": { "state": "admitted" }
}
```

Sources may request acknowledgement depth via `qos`; by default targets acknowledge `admitted` and the terminal state.



Figure 2: Core Message Lifecycle

2.5 Correlation and Idempotency

`correlation` links related messages into a conversation:

- A `response` carries the `id` of its `query` in `correlation`.
- An `ack` or `event` carries the `id` of its command in `correlation`.
- A committed objective that expands into tasks records its `id` as the tasks' `correlation`.

Targets keep a bounded **dedup window** keyed by `(source, id)` so re-delivered commands are recognized and not re-executed. The window duration is a target capability and is advertised during discovery.

2.6 Versioning

`roc` is `MAJOR.MINOR`:

- A **major** change alters envelope or lifecycle semantics and is not backward compatible.
- A **minor** change adds optional fields, verbs, or states and remains backward compatible.

Receivers MUST reject a major version they do not implement, and MUST ignore unknown fields and unknown `meta` entries within a supported major version.

2.7 Minimal Core Conformance

A device that implements only Core must be able to:

1. Decode an envelope, validate required fields, and enforce `roc` compatibility.
2. Evaluate authority, `priority`, `ttl`, and `deadline` at admission.
3. Deduplicate by `(source, id)`.
4. Emit `ack` and terminal `event` messages.
5. Preserve unknown payloads opaquely when acting as a router.

Full behavior for each plane is specified in the chapters that follow.

3 OpenROC Action Stack

3.1 Overview

The **OpenROC Action Stack** is the reasoning-to-work plane of OpenROC. It carries **objectives** — *why*, and under what constraints — and expands them into **actions**: tasks, skills, movements, and trajectories that a specific machine can execute. Actions in turn expand into **Actuation** commands (see the **Control Stack**), which realize motion at the hardware.

Keeping intent separate from hardware is deliberate: a new planning model can replace an old one without touching motion code, safety systems can reason about objectives rather than joint angles, and a single objective can fan out into many robot-specific actions.

3.2 The Action Plane

Plane	Question	Output
`action`	*What to do, and why?*	Objectives, tasks, skills, trajectories.

Data flows down (intent to work) and feedback flows up (outcomes close the loop). The boundary between the Action plane and the Control Stack is a committed objective: the point at which a goal is authorized and expanded into actions.

3.3 Objective

An **objective** is the outcome the system should achieve, expressed in domain terms:

```
{
```



Figure 3: Action Stack

```
  "objective": {
    "kind": "transport",
    "from": "bin.a",
    "to": "pallet.p3",
    "item": "red_box",
    "quantity": 1
  }
}
```

Objectives are declarative. They describe a target state of the world, not a procedure.

3.3.1 Goal and Success Criteria

An objective may attach machine-checkable **success criteria** so completion is objective rather than inferred:

Criterion	Example
`state_predicate`	`pallet.p3.contains(red_box)== true`
`tolerance`	`final_pose.distance(target)<= 0.005 m`
`deadline`	`completed_before("2026-10-13T09:01:00Z")`

Criterion	Example
`metric`	`energy_used_j <= 500`

3.3.2 Constraints

Constraints bound the solution space and are enforced downstream by Action and Actuation:

```
{
  "constraints": {
    "workspace": "cell.weld-2.workspace",
    "max_speed": "1.5 m/s",
    "max_payload": "5 kg",
    "avoid": ["human_zone"],
    "min_battery_pct": 20
  }
}
```

3.3.3 Authority and Confidence

An objective declares **who authorized it** and **how confident** the reasoning system is. Both are first-class so that safety logic and human-in-the-loop policies can act on them:

```
{
  "authority": { "principal": "planner.workcell-7", "role": "autonomous" },
  "confidence": 0.92,
  "requires_confirmation": false
}
```

Role	Meaning
`autonomous`	May be executed without human confirmation, subject to policy.
`supervised`	Requires an operator to confirm before dispatch.
`advisory`	Never executed directly; produces recommendations only.

Role	Meaning
`override`	Human-issued; supersedes autonomous objectives.

3.4 Action Verbs

Action messages use the Core envelope with `plane: "action"`.

Verb	Direction	Purpose
`propose`	planner → system	Submit a candidate objective for evaluation.
`commit`	planner → system	Authorize execution of a proposed objective.
`revise`	planner → system	Update an objective or constraints of a live action.
`abort`	planner/human → system	Withdraw an objective and, by default, its descendant actions.
`query_state`	any → planner	Ask for the current action context.
`execute`	system → robot	Run a skill or task graph.
`trajectory`	system → robot	Stream or load a trajectory.
`pause`	system → robot	Halt motion while keeping state.
`resume`	system → robot	Continue from a paused state.
`cancel`	system → robot	Abort the action and, by default, its descendants.
`event`	system → planner	Report lifecycle, progress, faults, and outcomes.

3.5 Action Lifecycle

Action messages follow the Core lifecycle, with these mappings:

1. **Propose.** The planner emits a `propose` with objective, constraints, authority, and confidence.
2. **Evaluate.** Policy checks authority, confidence thresholds, resource availability, and safety constraints.
3. **Commit or Reject.** A `commit` authorizes expansion into actions. A rejected proposal carries a reason code.
4. **Expand.** The system translates the committed objective into one or more tasks, skills, and trajectories, and topologically orders any task graph.
5. **Execute.** Each node is expanded into one or more Actuation commands; the robot emits progress events keyed by node `id` and the action `id`.
6. **Monitor.** The planner receives `event` messages as descendant actions progress or fail.
7. **Terminate.** The action ends as `completed`, `failed`, `cancelled`, `preempted`, or `expired`. `on_error` determines whether a failing node aborts the graph or triggers a recovery branch.

3.6 The Action Loop

A typical closed loop looks like this:

```
// 1. Planner proposes
{
  "roc": "1.0", "id": "01J8ZA...", "ts": "2026-10-13T09:00:00Z",
  "type": "command", "plane": "action", "verb": "propose",
  "source": "planner.workcell-7", "target": "cell.weld-2",
  "priority": "operational",
  "payload": {
    "objective": { "kind": "transport", "from": "bin.a", "to": "pallet.p3",
                   "item": "red_box" },
    "constraints": { "avoid": ["human_zone"], "max_payload": "5 kg" },
    "authority": { "principal": "planner.workcell-7", "role": "autonomous" },
    "confidence": 0.92
  }
}

// 2. System commits
{
  "roc": "1.0", "id": "01J8ZB...", "ts": "2026-10-13T09:00:00.050Z",
  "type": "ack", "plane": "action", "verb": "ack",
  "source": "cell.weld-2", "target": "planner.workcell-7",
  "priority": "operational", "correlation": "01J8ZA...",
  "payload": { "state": "admitted", "action": "commit" }
}
```

3.7 Human-in-the-Loop

Policy can require confirmation for specific objectives, confidence ranges, or roles. When an action requires confirmation, the system enters a **pending** condition and emits an event:

```
{
  "type": "event", "plane": "action", "verb": "event",
  "payload": {
    "state": "requires_confirmation",
    "reason": "confidence_below_threshold",
    "confidence": 0.61,
    "threshold": 0.80
  }
}
```

A human `override` objective — or an explicit `commit` — releases the hold. If no confirmation arrives before the action `deadline`, the action expires and any reserved resources are released.

3.8 Policy

Objectives are filtered by a **policy** that maps objectives and context to allow/deny/confirm outcomes. Policy is expressed separately from the objective itself so it can be owned by safety and operations teams:

```
policy:
  - match: { objective.kind: transport, authority.role: autonomous }
    require: { confidence: ">= 0.80" }
    action: allow
  - match: { constraints.avoid: ["human_zone"] }
    require: { interlock: "human_zone_clear" }
    action: confirm
  - match: { objective.kind: weld }
    action: deny_if: "authority.role == advisory"
```

Policy evaluation is deterministic and logged with the action so every outcome is auditable.

3.9 Action Feedback

The Action plane consumes upward feedback rather than raw sensor data:

- terminal events from descendant actions (`completed`, `failed`, `preempted`);

- resource events (battery, payload, thermal) delivered as `telemetry`;
- safety events (interlock opened, protective stop) delivered as `event`.

This closes the reasoning loop without requiring the planner to know how motion is produced.

3.10 Task and Skill

An action is built from **skills** — reusable, parameterized capabilities exposed by a robot (see **OpenROC Adapters** for capability discovery). A skill is named and typed:

Skill	Description	Typical Parameters
`pick`	Grasp an object	`object`, `grasp_pose`, `force`
`place`	Release an object at a target	`target`, `release_mode`
`move_to`	Move the end effector to a pose	`pose`, `frame`, `speed`
`navigate`	Drive a mobile base to a location	`goal`, `path_mode`
`follow`	Track a trajectory	`trajectory`, `blend`
`wait`	Hold until a condition or time	`condition`, `duration`
`home`	Return to a reference pose	`home_id`

3.10.1 Task Graph

Multiple skills compose into a **task graph** — a directed acyclic graph with explicit dependencies, conditions, and error handling. The graph is the unit of scheduling and cancellation.

```
{
  "task_graph": {
    "id": "pick_place_001",
    "nodes": [
      { "id": "approach", "skill": "move_to", "args": { "pose": "above(bin.a.
         red_box)" } },
      { "id": "grasp",    "skill": "pick",    "args": { "object": "red_box", "
         force": "20 N" },
        "after": ["approach"] },
      { "id": "transfer", "skill": "move_to", "args": { "pose": "above(pallet.p3)"
          },
        "after": ["grasp"] },
```

```
      { "id": "release",  "skill": "place",   "args": { "target": "pallet.p3" },
        "after": ["transfer"] }
    ],
    "on_error": "abort"
  }
}
```

3.10.2 Movement and Trajectories

Movement is described by **motion descriptions** and **trajectories**. A trajectory is a time-parameterized path through space, expressed in a named reference frame.

Waypoint Trajectory

```
{
  "skill": "follow",
  "trajectory": {
    "type": "waypoints",
    "frame": "robot.arm-1.base",
    "space": "cartesian",
    "waypoints": [
      { "pose": { "x": 0.45, "y": 0.10, "z": 0.30,
                  "qx": 0.0, "qy": 1.0, "qz": 0.0, "qw": 0.0 },
        "speed": "0.25 m/s" },
      { "pose": { "x": 0.50, "y": 0.10, "z": 0.20,
                  "qx": 0.0, "qy": 1.0, "qz": 0.0, "qw": 0.0 },
        "speed": "0.10 m/s" }
    ],
    "blend_radius": "0.02 m"
  }
}
```

Joint-Space Trajectory

```
{
  "skill": "follow",
  "trajectory": {
    "type": "joint_stream",
    "frame": "robot.arm-1",
    "space": "joint",
    "points": [
      { "t": 0.0,  "q": [0.0, -1.57, 0.0, 0.0, 0.0, 0.0] },
      { "t": 0.5,  "q": [0.1, -1.50, 0.0, 0.0, 0.0, 0.0] },
      { "t": 1.0,  "q": [0.2, -1.40, 0.0, 0.0, 0.0, 0.0] }
```

```
    ],
    "interpolation": "cubic"
  }
}
```

Motion Modifiers

Trajectories may carry modifiers that constrain how they are realized:

Modifier	Meaning
`frame`	Reference frame for the path.
`space`	`cartesian` or `joint`.
`blend_radius`	Rounding between consecutive segments.
`speed` / `accel`	Scalar scaling of the path profile.
`time_scale`	Global time scaling (1.0 = nominal).
`force_limit`	Maximum contact force during execution.
`cartesian_limit`	Bounds on deviation from the path.
`stop_condition`	Predicate that halts the trajectory early.

3.10.3 Execution Modes

Mode	Semantics
`atomic`	The action succeeds or fails as a whole; partial effects are avoided where possible.
`streamed`	Trajectory points are sent continuously and may be interrupted.
`supervised`	Execution pauses at checkpoints for confirmation.
`opportunistic`	Best-effort; may be preempted freely.

3.10.4 Time and Frames

Every action references a **frame** for spatial data and a **time base** for temporal data:

- Frames form a tree rooted at a world frame; each robot publishes its kinematic chain during discovery.
- Temporal fields use the action's `time_base`, normally `utc` or `monotonic`. Trajectory `t` values are relative to execution start unless an absolute `start_time` is given.

Frames and reference points are resolved by the Action interpreter or the Adapter that owns the target robot; OpenROC does not mandate a specific transform library, only that frame names are consistent between discovery and execution.

3.11 From Action to Actuation

The Action Stack never manipulates actuators directly. Each executing action node produces Actuation commands — position, velocity, torque, or impedance setpoints — that realize the intended motion. This separation means the same `follow` trajectory can run on different hardware by swapping the Actuation profile, and safety limits can be enforced at both the Action and Actuation layers.

3.12 Conformance Notes

An OpenROC Action implementation must validate objectives, constraints, skills, and trajectories against the registered schema and advertised capabilities, enforce authority and confidence policy at commit, maintain task-graph dependency ordering, expand nodes into correlated Actuation commands, enforce pause/resume/cancel semantics, and emit progress, lifecycle, and terminal events with reason codes. Implementations that only execute objectives (for example, a cell controller that accepts `commit` from a planner) need not implement `propose`; simpler profiles (for example, a smart gripper) may implement a reduced skill set.

4 OpenROC Control Stack

4.1 Overview

The **OpenROC Control Stack** is the plane of *how*, realizing the OpenROC **Actuation** plane. It carries device-level control to motors, servos, grippers, valves, and other physical transducers. Control is the point at which digital commands become force, motion, heat, and light — and therefore the point at which safety, determinism, and timing are most critical.

Control messages are the narrowest and most frequent in OpenROC. They are optimized for bounded latency and predictable framing, and they carry explicit control modes, units, and limits so that a driver can act on them without inference. On the wire these messages identify themselves with `plane: "actuation"`.

4.2 Concepts

4.2.1 Actuator

An **actuator** is the addressed unit of control. It is identified by an OpenROC address and described by a capability record discovered at connect time:

```
{
  "address": "robot.arm-1.joint-3",
  "kind": "servo",
  "modes": ["position", "velocity", "torque", "impedance"],
  "limits": {
    "position": { "min": -3.14, "max": 3.14, "unit": "rad" },
    "velocity": { "max": 6.28, "unit": "rad/s" },
    "torque":   { "max": 40.0, "unit": "N*m" },
    "current":  { "max": 12.0, "unit": "A" }
  },
  "rate_hz": 1000,
  "units": { "position": "rad", "velocity": "rad/s" }
}
```

4.2.2 Control Modes

Mode	Setpoint	Use
`position`	Target position	Point-to-point moves, servo loops.
`velocity`	Target velocity	Conveyors, wheels, spindles.
`torque`	Target torque/current	Force control, haptics.
`impedance`	Stiffness/damping + target	Compliant manipulation.
`pwm`	Raw duty cycle	Simple drivers, lighting, fans.
`discrete`	On/off/open/close	Relays, grippers, valves.

The active mode determines which fields of a setpoint are meaningful. A driver MUST reject a setpoint whose mode it does not support or whose values exceed advertised limits.

4.2.3 Setpoint

A **setpoint** is the core Actuation payload:

```
{
  "mode": "impedance",
  "values": { "position": 1.20, "stiffness": 120.0, "damping": 8.0 },
  "units": { "position": "rad", "stiffness": "N*m/rad", "damping": "N*m*s/rad" },
```

```
  "frame": "robot.arm-1.joint-3",
  "t": "2026-10-13T09:00:01.250Z"
}
```

Values may be **immediate** (apply now) or **scheduled** by timestamp for time-synchronized coordinated motion.

4.3 Message Verbs

Actuation messages use the Core envelope with `plane: "actuation"`.

Verb	Direction	Purpose
`set`	system → device	Apply a single control mode and setpoint.
`stream`	system → device	Send a time-indexed sequence of setpoints.
`enable`	system → device	Energize the actuator and enter closed-loop control.
`disable`	system → device	De-energize; allow the actuator to freewheel.
`brake`	system → device	Engage a holding brake.
`home`	system → device	Execute a homing/calibration sequence.
`estop`	system → device	Enter protective stop state immediately.
`feedback`	device → system	Report state, limits, faults, and derived quantities.

4.3.1 Set Example

```
{
  "roc": "1.0",
  "id": "01J8ZC...",
  "ts": "2026-10-13T09:00:01.250Z",
  "type": "command",
  "plane": "actuation",
```

```
  "verb": "set",
  "source": "robot.arm-1.controller",
  "target": "robot.arm-1.joint-3",
  "priority": "operational",
  "deadline": "2026-10-13T09:00:01.260Z",
  "payload": {
    "mode": "torque",
    "values": { "torque": 4.2 },
    "units": { "torque": "N*m" },
    "ramp": "10 ms"
  }
}
```

4.4 Streaming Control

High-rate control uses `stream`. Each message carries a short, time-indexed window of setpoints so the device has work buffered ahead of time:

```
{
  "verb": "stream",
  "payload": {
    "mode": "position",
    "rate_hz": 500,
    "start_time": "2026-10-13T09:00:02.000Z",
    "samples": [
      { "dt": 0.002, "position": 1.200 },
      { "dt": 0.002, "position": 1.205 },
      { "dt": 0.002, "position": 1.210 }
    ],
    "underrun_policy": "hold_last"
  }
}
```

`underrun_policy` defines behavior if the device exhausts its buffer before new samples arrive: `hold_last`, `decelerate`, or `stop`. Streaming messages are typically encoded in **OpenROC-BIN** to keep latency and bandwidth low.

4.5 Timing and Determinism

Actuation exposes timing properties explicitly so consumers can reason about real-time behavior:

Property	Meaning
`rate_hz`	Nominal control-loop rate.
`jitter_us`	Guaranteed maximum timing jitter.
`latency_us`	Maximum command-to-actuation latency.
`scheduling`	`cyclic`, `event`, or `best_effort`.
`sync_group`	Name of the time-synchronization group.

Members of a `sync_group` share a time base so multiple actuators can be scheduled to change state on the same instant — the basis for coordinated multi-axis motion.

4.6 Feedback

Devices emit `feedback` events at a configurable rate. Feedback is the ground truth for closed-loop control and safety monitoring:

```
{
  "type": "telemetry",
  "plane": "actuation",
  "verb": "feedback",
  "payload": {
    "position": 1.203,
    "velocity": 0.41,
    "torque": 4.18,
    "current": 2.7,
    "temperature_c": 48.2,
    "mode": "torque",
    "limits_active": [],
    "faults": []
  }
}
```

Feedback frequency, fields, and units are declared in the actuator capability record, so consumers can subscribe precisely.

4.7 Safety Integration

Actuation is where OpenROC's safety model becomes physical:

- Every actuator enforces its advertised **limits** locally, independent of higher planes.
- `estop` is terminal and may be issued by any `emergency`-priority source; it preempts all other commands to that device.



Figure 4: Control Stack

- A `feedback` fault transitions the actuator to a safe state according to its **fault policy** (`hold`, `disable`, `brake`, `retract`).
- Watchdogs require periodic `set`/`stream` traffic; absence of traffic triggers the configured **watchdog policy**.
- Actuation commands carry the safety class and authority of their originating Action chain for audit.

4.8 Pipeline

An Actuation command flows through a fixed pipeline on the device:

1. **Decode** the envelope and setpoint (BIN or JSON).
2. **Admit** — check authority, priority, deadline, and watchdog state.
3. **Clamp** values to advertised limits.
4. **Convert** units to the device's native representation.
5. **Schedule** on the control loop, synchronized via `sync_group`.
6. **Apply** the setpoint and update feedback.
7. **Report** feedback and terminal events upward.

4.9 Conformance Notes

A Control Stack implementation must advertise capabilities and limits through discovery, enforce mode and limit checks, honor `emergency`-priority `estop`, implement its declared `underrun_policy` and watchdog policy, and emit feedback and terminal events. Implementations are encouraged to use OpenROC-BIN on control paths to meet their declared timing properties.

5 OpenROC-JSON

5.1 Overview

OpenROC-JSON is the human-readable encoding of OpenROC. It represents every Core envelope, plane payload, and lifecycle event as UTF-8 JSON. It is the encoding used for development, debugging, configuration files, tooling, test vectors, and any link where readability and toolability matter more than raw bytes.

OpenROC-JSON is not "any JSON that happens to look right." It is a strict profile with normative rules for field naming, numbers, units, time, and binary data, so that two conformant implementations always produce byte-identical canonical output.

5.2 Design Rules

1. **Encoding.** Messages are UTF-8 JSON objects. A message is exactly one JSON value, optionally followed by a single newline.
2. **Field names.** Object keys use `snake_case` and are case-sensitive.
3. **Envelope.** Every message contains the Core envelope fields (`roc`, `id`, `ts`, `type`, `plane`, `verb`, `source`, `target`, `priority`) plus `payload`.
4. **Unknown fields.** Receivers ignore unknown fields within a supported `roc` major version.
5. **Canonical form.** Canonical output orders keys lexicographically, uses no insignificant whitespace, and emits numbers per the rules below.
6. **No duplicate keys.** Duplicate object keys are invalid.
7. **Extensions.** Implementation-specific data lives under `meta` and must not collide with reserved names.

5.3 Numbers

JSON numbers are mapped to OpenROC scalar types deterministically:

Type	Rule
Integers	Emitted without a decimal point or exponent. Range up to 2^53 in JS-safe mode.
Floats	Emitted in shortest round-trippable decimal form; no trailing zeros.
Large integers	Emitted as strings when they exceed the safe integer range.
Non-finite	`NaN` and `Infinity` are not allowed.

Any physical quantity should carry an explicit **unit** rather than relying on convention:

```
{ "distance": { "value": 0.45, "unit": "m" } }
```

For compactness, arrays of like quantities may share a `units` object:

```
{
  "values": [0.45, 0.50, 0.55],
  "units": { "values": "m" }
}
```

5.4 Time

All timestamps are RFC 3339 strings in UTC with exactly millisecond precision:

```
"ts": "2026-10-13T09:00:00.000Z"
```

Durations use ISO 8601 duration strings:

```
"ttl": "PT2S"
```

Monotonic time, when needed, is an object that names the clock and a value:

```
"t": { "clock": "monotonic", "value": 1284500, "unit": "us" }
```

5.5 Binary Data

Binary payloads (for example, a compact trajectory blob or a signature) are encoded with **base64url without padding** and tagged with a `format`:

```
{
  "trajectory_blob": {
    "format": "openroc-bin",
    "encoding": "base64url",
    "data": "A2Z4_..."
  }
}
```

5.6 Canonical Example

A complete OpenROC-JSON command:

```
{
  "deadline": "2026-10-13T09:00:05.000Z",
  "id": "01J8Z9Q7V6B4M2T0R3C5K8P1XA",
  "payload": {
    "args": { "object": "red_box", "force": { "unit": "N", "value": 20 } },
    "skill": "pick"
  },
  "plane": "action",
  "priority": "operational",
  "roc": "1.0",
  "source": "planner.workcell-7",
  "target": "robot.arm-1",
  "ts": "2026-10-13T09:00:00.000Z",
  "type": "command",
  "verb": "execute"
}
```

Its canonical form orders keys lexicographically and removes all non-significant whitespace.

5.7 Schemas

Every verb and payload has a **JSON Schema** (2020-12) published as part of the OpenROC schema set. Schemas are the source of truth for validation and code generation:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
```



Figure 5: JSON Encoding

```
  "$id": "https://openroc.org/schemas/action/execute/1.0.json",
  "type": "object",
  "required": ["skill"],
  "properties": {
    "skill": { "type": "string" },
    "args": { "type": "object" }
  },
  "additionalProperties": false
}
```

Schemas are versioned alongside `roc` and referenced by an `$id` derived from the plane, verb, and version.

5.8 Streaming

Newline-delimited JSON (NDJSON) is the standard streaming form for OpenROC-JSON: one canonical message per line. This suits logging, replay, and low-rate telemetry. For high-rate or bandwidth-constrained links, use **OpenROC-BIN** instead.

5.9 Interoperability with BIN

OpenROC-JSON and OpenROC-BIN are semantically identical. A conformant implementation can convert losslessly in both directions:

- **JSON → BIN** encodes the same fields using opcodes and packed values.
- **BIN → JSON** emits canonical JSON, including units and time in the forms above.

The conformance suite includes **round-trip vectors** that assert `json -> bin -> json` yields the canonical input.

5.10 Validation Rules

A validator must reject a message when:

- a required envelope or payload field is missing;
- a field has the wrong type or an unknown verb for its plane;
- a number is non-finite or violates a documented range;
- a timestamp or duration is not in the normative form;
- base64 data is not valid base64url or exceeds a declared size limit.

Implementations SHOULD produce descriptive validation errors that name the offending JSON Pointer.

5.11 Conformance Notes

An OpenROC-JSON implementation must parse and emit UTF-8 JSON, honor the number, time, unit, and binary rules above, validate against the published schemas, and support canonical output. Canonicalization is required for signatures and digests so that integrity checks are stable across implementations.

6 OpenROC-BIN

6.1 Overview

OpenROC-BIN is the compact binary encoding of OpenROC. It represents the same semantic model as **OpenROC-JSON** but in a form optimized for low latency, low bandwidth, minimal decode cost, and deterministic framing. It is the preferred encoding for high-rate control streams, constrained links, and safety-critical paths.

OpenROC-BIN is designed so a small device can decode a command with a fixed, bounded amount of work and no dynamic allocation. Every message is framed, self-describing, and checksummed.

6.2 Framing

A BIN **frame** is a header followed by a payload and a trailer. Fields are little-endian.

```
+--------+--------+--------+----------------+--------+---------+
| MAGIC  | VER    | FLAGS  | LENGTH         | HEADER | PAYLOAD | CRC32 |
| 2B     | 1B     | 1B     | varint         | ...    | ...     | 4B    |
+--------+--------+--------+----------------+--------+---------+
```

Field	Size	Description
`MAGIC`	2 B	`0x4F 0x52` (`OR`). Identifies an OpenROC frame.
`VER`	1 B	Frame format version (`0x01`).
`FLAGS`	1 B	Bit flags: compressed, signed, fragmented, has-integrity.
`LENGTH`	varint	Length in bytes of `HEADER + PAYLOAD`.
`HEADER`	varint	Encoded Core envelope as opcode/value pairs.
`PAYLOAD`	bytes	Plane-specific body.
`CRC32`	4 B	CRC-32C over `HEADER + PAYLOAD`.

Optionally, when `FLAGS.signed` is set, a fixed-size signature block precedes the CRC.

6.3 Varints

Unsigned integers use **LEB128** varints: seven bits per byte, high bit set to continue, least-significant group first. Zig-zag encoding is used for signed values. Varints keep small common values to a single byte while allowing full-width values.

6.4 The Field Table

Within `HEADER`, fields are encoded as **tag/value** pairs. Each tag is a varint opcode. Tags are grouped by scope:

Tag Range	Scope
`0x00-0x1F`	Core identity and timing (`id`, `ts`, `roc`)
`0x20-0x3F`	Addressing (`source`, `target`)
`0x40-0x5F`	Classification (`type`, `plane`, `verb`, `priority`)
`0x60-0x7F`	Lifecycle (`deadline`, `ttl`, `correlation`, `sequence`, `qos`)
`0x80-0xFF`	Plane payload opcodes

Tags are ordered so the most common fields appear first, improving cache locality during decode.

6.5 Strings and Common Values

- Strings are length-prefixed UTF-8 blobs.
- Frequently repeated strings have **dictionary opcodes**: `verb` and `priority` values, `type`, and `plane` are encoded as single-byte constants.

Enum	Values (opcode order)
`type`	command, event, query, response, telemetry, ack
`plane`	action, actuation
`priority`	emergency, safety, operational, best_effort
`mode`	position, velocity, torque, impedance, pwm, discrete

6.6 Payload Encoding

Payloads use compact records rather than schemas-at-runtime. Each plane defines its payload layout in the OpenROC schema set, which also generates the BIN codec:

```
Action.Execute:
  skill        varint (skill id from capability handshake)
  arg_count    varint
  args[]       tag/value pairs (typed)
```

Because skill and actuator identifiers are negotiated during the capability handshake (see **OpenROC Adapters**), they are transmitted as small integers instead of strings, which greatly reduces message size for repeated calls.

6.7 Example

The OpenROC-JSON `set` command from the Actuation chapter encodes to roughly the following BIN layout:

```
4F 52 01 00 1E                    ; magic, ver, flags, length=30
  05  <8B id>                    ; tag 0x05: id
  07  <varint ts>                ; tag 0x07: ts (deltas from epoch)
  40  01                          ; type = command
  41  02                          ; plane = actuation
  42  06                          ; verb = set
  43  02                          ; priority = operational
  20  <dotted source>             ; tag 0x20: source
  21  <dotted target>             ; tag 0x21: target
  60  <varint deadline>           ; tag 0x60: deadline
  80  02                          ; tag 0x80: mode = torque
  81  3C                         ; tag 0x81: torque = 4.2 (Q16.16)
  <CRC32C>
```

Scripts and test vectors in the conformance suite pin these bytes exactly.

6.8 Numeric Encoding

Physical values are encoded as scaled integers to avoid floating-point ambiguity:

Type	Encoding
Position (rad)	Q16.16 fixed-point
Velocity (rad/s)	Q16.16
Torque (N·m)	Q16.16
Current (A)	Q8.8
Temperature (°C)	Q8.8
Percent / duty	Q0.16

The scale and unit for each field are fixed by the schema, so no unit bytes are transmitted. Where higher range or precision is required, a schema may select float32 or float64.



Figure 6: BIN Frame Layout

6.9 Fragmentation and Compression

- **Fragmentation.** Frames larger than a link MTU set `FLAGS.fragmented`; each fragment carries a fragment index and count, and the receiver reassembles before CRC verification.
- **Compression.** `FLAGS.compressed` indicates the payload is DEFLATE-compressed. Compression is optional and never applied to safety-critical control frames.

6.10 Checksums and Integrity

Every frame carries a CRC-32C for error detection. When `FLAGS.signed` is set, the frame also carries a signature over the canonical semantic content — the same content that **OpenROC-JSON** canonicalization would produce — so an integrity check is stable across encodings. Signature algorithms and key references follow the Security & Safety chapter.

6.11 Transport Independence

BIN frames are self-delimiting and may be carried over any transport: a byte stream, a CAN window, an EtherCAT datagram, a UDP message, or a shared-memory ring. When a transport has its own framing (for example CAN), adapters may omit the `MAGIC`/`LENGTH` fields and map them to the transport, but the CRC and semantic content are unchanged.

6.12 Conformance Notes

An OpenROC-BIN implementation must parse and emit framed messages with valid `MAGIC`, version, length, and CRC-32C; implement LEB128 varints; honor dictionary and payload opcodes; and support lossless round-trip conversion with OpenROC-JSON. Decoders MUST reject frames with a bad CRC, an unsupported version, or an out-of-range length, and MUST apply bounded resource limits while parsing untrusted frames.

7 OpenROC Adapters

7.1 Overview

OpenROC Adapters connect OpenROC to the robots, actuators, and middleware that already exist. They translate OpenROC messages into native protocols — ROS 2, CAN, CANopen, EtherCAT, Modbus, MQTT, gRPC, OPC UA, and vendor SDKs — and translate native feedback back into OpenROC events. Adapters make OpenROC an **overlay** on existing infrastructure rather than a requirement to replace it.

An adapter is the trust and capability boundary of a deployment: it is where OpenROC's abstract addresses and planes meet concrete buses, endpoints, and device models.

7.2 The Adapter Contract

Every adapter implements the same three responsibilities:

1. **Discovery.** Publish the capabilities it exposes as OpenROC endpoint descriptors.
2. **Translation.** Map OpenROC envelopes and plane payloads to and from native operations.
3. **Feedback.** Convert native status, faults, and telemetry into OpenROC `feedback` and `event` messages.

Adapters are bidirectional but need not be symmetric: a read-mostly telemetry adapter may implement only feedback and queries.

7.2.1 Endpoint Descriptor

An adapter declares an endpoint for each addressable capability:

```
{
  "address": "robot.arm-1",
  "adapter": "vendor://acme/arm-driver@2.3",
  "planes": ["action", "actuation"],
  "skills": [
    { "id": 1, "name": "move_to", "args_schema": "action/move_to/1.0" },
    { "id": 2, "name": "pick",    "args_schema": "action/pick/1.0" }
  ],
```



Figure 7: Adapter Topology

```
  "actuators": [
    { "address": "robot.arm-1.joint-3", "kind": "servo",
      "modes": ["position", "velocity", "torque"] }
  ],
  "encodings": ["json", "bin"],
  "qos": { "ordering": "per_source", "max_latency_ms": 10 },
  "dedup_window": "PT5S"
}
```

Skill and actuator identifiers are **numeric** so OpenROC-BIN can address them compactly after the handshake.

7.3 Capability Discovery

Adapters support two discovery models:

- **Push.** On connect, the adapter emits an endpoint descriptor advertisement.
- **Pull.** A consumer sends a `query` for capabilities and receives a `response`.

Descriptors are versioned and re-advertised when capabilities change. Consumers cache descriptors and reference capabilities by their numeric ids until an update arrives. This handshake is also where encodings, QoS guarantees, dedup windows, and sync groups are negotiated.

7.4 Common Adapters

Adapter	Native Protocol	Mapping Highlights
ROS 2	DDS topics/services/actions	OpenROC Action → ROS 2 action; Actuation → `ros2_control`; telemetry → topics.
CAN / CANopen	CAN frames, PDO/SDO	Actuation → PDOs; feedback → TPDOs; `estop` → RPDO or bus broadcast.
EtherCAT	Cyclic process data	Actuation → process-data image; sync via distributed clocks.
Modbus	Registers, coils	Actuation → register writes; feedback → register reads.
MQTT	Publish/subscribe topics	Core envelopes on topics; QoS mapped to MQTT levels.
gRPC	Unary/stream RPC	Envelopes as protobuf `Any`; streaming control as server streams.
OPC UA	Nodes, methods, subscriptions	Endpoints as nodes; feedback as subscriptions.
Vendor SDK	Proprietary	Custom mapping, but must expose OpenROC descriptors and lifecycle.

An adapter may bridge multiple native protocols to one OpenROC address, presenting a unified capability set.

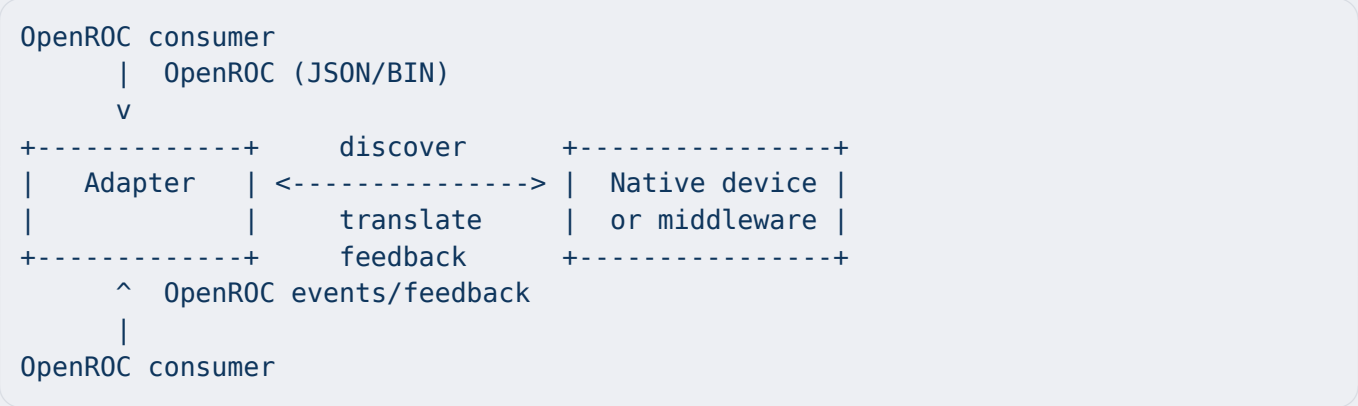
7.5 Translation Rules

Adapters follow these rules to preserve OpenROC semantics:

1. **Envelope preserved.** Core fields are never lost; when a native protocol cannot carry a field, the adapter records it in the endpoint's `meta` capability and reports the loss.

2. **Lifecycle mapped.** Admission, execution, and terminal states map to the native protocol's closest equivalents; where none exists, the adapter synthesizes them.
3. **Idempotency enforced.** The adapter maintains the dedup window on behalf of a target that cannot.
4. **Safety class honored.** `emergency` and `safety` messages map to native priority or safety mechanisms; if impossible, the adapter MUST reject rather than downgrade silently.
5. **Units converted.** Native units are converted to OpenROC base units at the boundary, using the actuator's declared conversions.
6. **Frames reconciled.** When a transport provides framing, the adapter maps OpenROC-BIN's `MAGIC`/`LENGTH` to it and preserves the CRC and semantic content.

7.6 Bidirectional Flow



The adapter is also a **filter**: it can rate-limit, clamp, or reject commands that violate native safety or capability constraints, reporting the reason with a standard code.

7.7 Connection and Reconnection

Adapters manage the lifecycle of native connections independently of OpenROC:

- On startup, the adapter connects, performs discovery, and advertises descriptors.
- On native failure, the adapter emits `event` messages with `state: degraded` for affected endpoints and applies each endpoint's **failure policy** (`hold`, `safe`, `retry`).
- On reconnect, the adapter re-runs discovery and re-advertises updated descriptors.

Consumers can subscribe to adapter health through a standard endpoint (`adapter.<name>.health`) that reports link state, latency, and error counters.

7.8 Addressing and Routing

An OpenROC **router** may sit between consumers and adapters. It resolves dotted addresses to the adapter that owns the matching prefix, forwards envelopes, and merges wildcard sub-

scriptions. Routers preserve the Core envelope, never rewrite `id`, and propagate `correlation` unchanged.

7.9 Security at the Boundary

Adapters participate in the security model (see **Security & Safety**):

- native connections are authenticated and, where possible, encrypted;
- adapter identity is bound to a key used to verify command signatures;
- the adapter enforces authority for its endpoints, so a command signed by an unauthorized principal is rejected before reaching native hardware.

7.10 Conformance Notes

An OpenROC Adapter must publish endpoint descriptors, implement the translation rules above, enforce idempotency and safety classes, and report feedback and health. An adapter that cannot preserve a required semantic must declare the limitation in its descriptor and reject incompatible messages instead of degrading them silently.

8 Security & Safety

8.1 Overview

OpenROC commands can move machines, apply force, and consume energy. A spoofed command or a delayed safety response can cause physical damage or injury. Security and safety are therefore foundational properties of the protocol, not optional layers: every OpenROC message carries identity, integrity, and classification so that a receiving device can decide whether it is allowed to act — and how quickly it must stop if it is not.

OpenROC separates the two concerns while binding them together:

- **Security** answers *who sent this, and has it been altered?*
- **Safety** answers *is it permitted to move, under what bounds, and what happens if something goes wrong?*

8.2 Identity and Authentication

Every OpenROC principal has an **identity** — a key pair and a logical role. Principals include planners, controllers, operators, adapters, and devices. Identity is established at connection time through mutual authentication:

- **Transport security.** Adapters and routers use mTLS or an equivalent authenticated channel. Device certificates chain to a deployment trust root.

- **Principal binding.** The authenticated channel is bound to an OpenROC principal, so `source` in an envelope is trustworthy.
- **Role.** Principals carry roles (`planner`, `controller`, `operator`, `device`, `auditor`) that scope their authority.

8.3 Command Integrity

Envelopes may carry an `integrity` block that signs the canonical message content:

```
{
  "integrity": {
    "alg": "ES256",
    "key_id": "planner.workcell-7#2026",
    "canonical": "openroc-json",
    "signature": "MEUCIQ..."
  }
}
```

Rules:

1. **Canonicalization first.** Signatures are computed over the canonical form of the message (see **OpenROC-JSON**), so a signature is valid regardless of the encoding used on the wire.
2. **Plane-aware coverage.** The signature covers the envelope and payload, excluding the `integrity` field itself.
3. **Algorithm agility.** `alg` names the signature algorithm; `key_id` identifies the key. Devices advertise accepted algorithms.
4. **Verification required for actuation.** Any command that can cause motion MUST be verified if the transport is not itself end-to-end trusted.

8.4 Replay Protection

OpenROC defends against replayed commands with three mechanisms:

- **Unique `id`.** Every message has a unique identifier; targets keep a bounded dedup window and reject repeated ids.
- **`ts` freshness.** Targets reject messages whose `ts` is outside an accepted skew window.
- **`sequence`.** Ordered streams carry a monotonic per-(source,target) counter; gaps or regressions are detected.

For `emergency` traffic, replay is also harmless by design because `estop` is idempotent: issuing it again simply reaffirms the safe state.

8.5 Safety Classes in Practice

The Core priority/safety classes drive enforcement at every layer:

Class	Requirement
`emergency`	Never dropped, never delayed, always admitted to the safe state.
`safety`	Admitted only if interlocks pass; may not be downgraded.
`operational`	Admitted subject to authority and resource checks.
`best_effort`	Droppable; never blocks the classes above.

A device MUST be able to receive and act on an `emergency` command even when it is saturated with `operational` traffic.

8.6 Interlocks and Envelope Limits

Interlocks are named conditions that gate execution. A command referencing an interlock is admitted only when the interlock is satisfied:

```
{
  "payload": {
    "skill": "move_to",
    "args": { "pose": "above(pallet.p3)" },
    "interlocks": ["human_zone_clear", "door_closed"],
    "limits": { "max_speed": "1.0 m/s", "max_force": "30 N" }
  }
}
```

Envelope limits bound motion independently of the commanded path. Limits are enforced at the Action and Actuation planes so that even a faulty action cannot exceed the machine's safe operating envelope.

8.7 Protective Stop

OpenROC defines a protective stop that is terminal and idempotent:

- `estop` (priority `emergency`) transitions the target to a safe state immediately.

- The safe state and stop category (for example, controlled deceleration vs. immediate power removal) are declared in the actuator capability record.
- `estop` propagates: a stop on a cell propagates to every descendant endpoint unless an endpoint explicitly declares isolation.
- Recovery requires an explicit `enable`/`reset` from an authorized principal and, by policy, manual confirmation.

E-stops implemented at the hardware level remain authoritative; OpenROC `estop` is a protocol-level complement that coordinates software-commanded machines.

8.8 Watchdogs

Every actuator and adapter has a watchdog. Consumers must refresh it with `set`/`stream` traffic within the declared interval; absence of traffic triggers the watchdog policy (`hold`, `disable`, `brake`, or `retract`). Watchdog expiry is reported as a safety event and is logged for audit.

8.9 Geofencing and Zones

Spatial and logical **zones** constrain where and when operations may occur:

- A zone is a named region with a policy (`allowed`, `forbidden`, `reduced_speed`).
- Commands that would move a machine into a forbidden zone are rejected or clamped.
- Zone membership is evaluated continuously from feedback, so an unexpected intrusion triggers a protective stop.

8.10 Rate Limiting and Command Budgets

To contain runaway loops and abuse, adapters and devices enforce **command budgets**: maximum command rates, maximum energy per interval, and maximum concurrent actions per target. Exceeding a budget produces a `rejected` outcome with a reason code, not a silent drop.

8.11 Audit and Traceability

Every admitted command, transition, interlock decision, and safety event is recorded with enough context to reconstruct the causal chain: `id`, `source`, authority, plane, verb, safety class, action correlation, and terminal state. Records are tamper-evident through signatures or an append-only ledger in regulated deployments.

8.12 Threat Model Summary

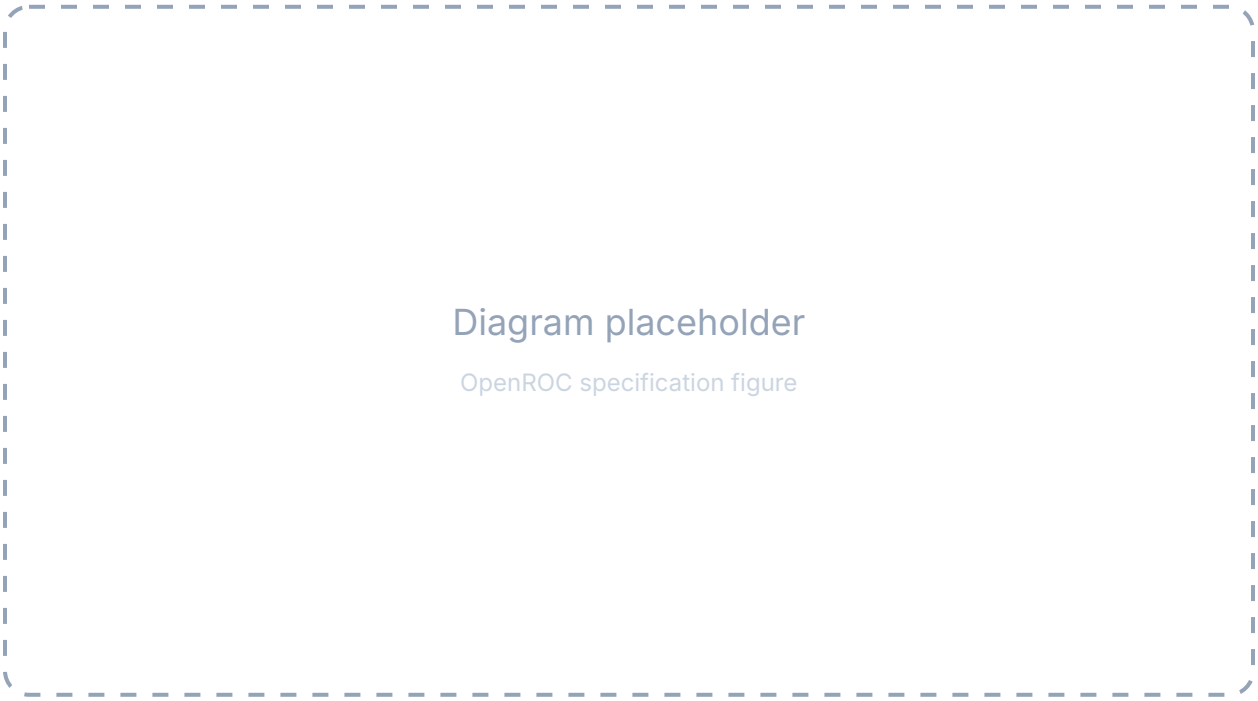


Figure 8: Safety Layers

Threat	Mitigation
Spoofed command	Mutual authentication + signed envelopes.
Modified command	Canonical signatures + CRC in BIN frames.
Replayed command	Unique ids, dedup window, `ts` freshness, sequence counters.
Unauthorized principal	Role- and address-scoped authority checks.
Runaway motion	Envelope limits, rate budgets, watchdogs, geofencing.
Supply-chain compromise	Signed firmware and adapter images, trust-root pinning.
Delayed stop	`emergency` class preemption and dedicated safety paths.

8.13 Conformance Notes

A security-enabled OpenROC implementation must authenticate principals, verify signatures on actuation-capable commands over untrusted links, enforce replay protections, and honor safety classes without downgrade. A safety-enabled implementation must declare safe states, implement protective stop and watchdog policies, enforce interlocks and envelope limits, and emit auditable safety events.

9 Testing & Validation

9.1 Overview

Because OpenROC spans AI reasoning, robot behavior, and physical actuation, it must be verified at every level: that encodings round-trip, that lifecycles behave, that planes compose, that adapters translate faithfully, and that safety properties hold. This chapter defines how OpenROC implementations are tested and how conformance is established.

Testing is organized around a shared **conformance test suite**, a set of **test vectors**, reference **simulators**, and an **interoperability matrix**.

9.2 Test Levels

Level	Scope	Environment
Encoding	JSON/BIN round-trip, canonicalization, CRC.	Host.
Unit	Verb validation, state machine, policy.	Host.
Plane	Action → Actuation composition.	Simulator.
Adapter	Native protocol translation.	Simulator + loopback.
Integration	Multi-endpoint tasks, QoS, preemption.	Simulator or lab.
Hardware-in-the-loop	Real controllers, real timing.	Bench.
Safety validation	Stops, interlocks, watchdogs, limits.	Bench + sim.



Figure 9: Test Pyramid

Level	Scope	Environment
Fuzzing	Malformed and adversarial input.	Host.

9.3 Test Vectors

The OpenROC schema set ships with normative **test vectors** under `test/vectors`. Each vector pairs an encoded message with a canonical semantic description so implementations can assert exact behavior:

```
{
  "name": "action.execute.pick.minimal",
  "json": {
    "roc": "1.0", "id": "01J8ZA...", "ts": "2026-10-13T09:00:00.000Z",
    "type": "command", "plane": "action", "verb": "execute",
    "source": "planner.workcell-7", "target": "robot.arm-1",
    "priority": "operational",
    "payload": { "skill": "pick", "args": { "object": "red_box" } }
  },
```

```
  "bin_hex": "4F520100...",
  "expected": { "canonical_hash": "sha256:..." }
}
```

Vectors cover:

- **Canonical form** — key ordering, number formatting, whitespace.
- **Round-trip** — `json -> bin -> json` equals canonical input.
- **Boundaries** — maximum/minimum values, varint edges, unit conversions.
- **Lifecycle** — each transition and terminal state.
- **Negatives** — messages that must be rejected, with reason codes.

9.4 Conformance Suite

An implementation is tested against the suite via a standard **test harness protocol**: the implementer provides a small shim that exposes the implementation to the harness, and the harness runs all vectors and behavioral tests for the declared capability profile.

```
openroc-conformance run \
  --impl ./target/release/my-controller \
  --profile actuation \
  --vectors ./openroc/test/vectors \
  --report conformance.json
```

The report records pass/fail per test, the implementation's declared profile, and any waivers. A capability profile must pass all **required** tests for that profile; optional tests affect maturity ratings, not conformance.

9.4.1 Behavioral Tests

Beyond static vectors, the suite drives the implementation through scripted scenarios:

1. Send a command and assert the full lifecycle of `ack`/`event` messages.
2. Re-deliver the same `id` and assert no duplicate effect.
3. Send an expired command and assert `rejected`/`expired`.
4. Send a higher-priority command and assert the lower-priority one is `preempted`.
5. Trigger an interlock failure and assert the command is rejected.
6. Let a watchdog expire and assert the configured safe behavior.

9.5 Simulation

Two reference simulators accelerate development:

- **OpenROC Sim** runs the Action plane against kinematic models with configurable timing, faults, and sensor noise. It is deterministic and replayable from a seed.

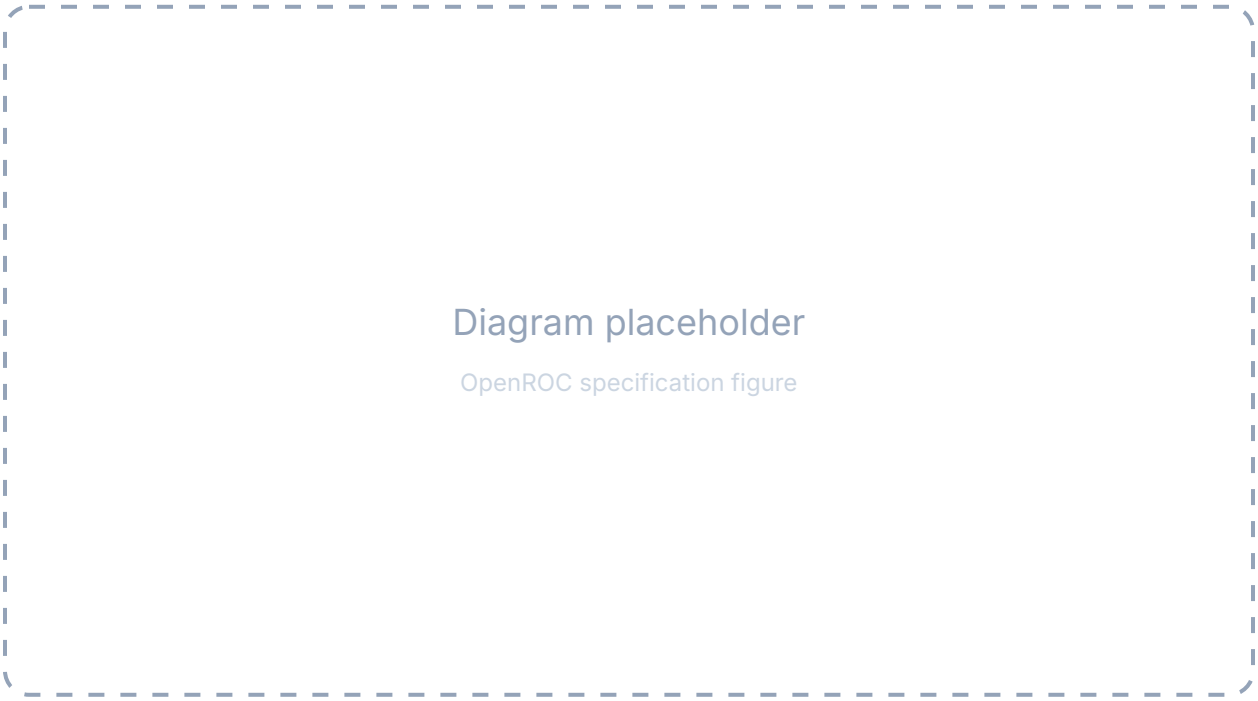


Figure 10: HIL Test Bench

- **Actuation Sim** models actuators with the same capability records and limits as real devices, including watchdog and underrun behavior.

Simulation lets the same conformance vectors run without hardware and makes regressions reproducible.

9.6 Hardware-in-the-Loop

HIL testing connects the real controller to a simulated plant (or the real plant to a simulated controller) to validate timing, synchronization, and stop behavior under realistic load. HIL is required for safety profiles:

- measure command-to-actuation latency and jitter against declared values;
- verify `estop` propagation and stop category;
- verify `sync_group` coordinated motion;
- verify watchdog and fault policies.

9.7 Fuzzing and Robustness

A conformant decoder must be robust to hostile input. The suite includes a fuzzing harness for OpenROC-BIN and OpenROC-JSON with these invariants:

- no crash, hang, or unbounded allocation on any input;

- bad CRC, bad length, and bad version are rejected;
- deeply nested or oversized payloads are rejected within bounded memory;
- error messages never leak secrets or uninitialized memory.

9.8 Interoperability Matrix

Conformance is necessary but not sufficient for interop. The suite publishes an **interop matrix** that records tested pairs of implementations across planes, encodings, and adapters:

Producer	Consumer	Planes	Encodings	Result
planner-ref	controller-ref	action	JSON	pass
controller-ref	arm-adapter	action→actuation	BIN	pass
planner-vendor-a	controller-ref	action	JSON/BIN	pass
arm-adapter	ros2-adapter	actuation	BIN	partial (rate limit)

Vendors are encouraged to publish their rows so integrators can predict behavior.

9.9 Continuous Integration

Recommended CI gates for an OpenROC implementation:

1. Lint and build the generated codecs from the published schemas.
2. Run encoding and unit tests on every commit.
3. Run the conformance suite for the declared profile.
4. Run the fuzzing harness for a bounded time budget.
5. Publish the conformance report and interop rows as artifacts.

9.10 Conformance Notes

A claim of OpenROC conformance must name the capability profile and link to a conformance report generated by the standard harness. Safety profiles additionally require HIL evidence for stop, watchdog, and timing behavior. Test vectors and the harness protocol are versioned with the protocol so that conformance remains meaningful as OpenROC evolves.

10 Deployment & Updates

10.1 Overview

OpenROC deployments are long-lived and heterogeneous: planners, controllers, adapters, and actuators from different vendors, at different versions, rebooted on different schedules. The

protocol therefore treats deployment and update as first-class concerns. Version negotiation, staged rollout, signed artifacts, and rollback are specified so that a portion of a system can be updated without breaking the whole.

10.2 Deployment Topology

A typical deployment has four kinds of roles:

Role	Examples	Update Cadence
Planner	AI model servers, workcell planners	Frequent, independent.
Controller	Robot controllers, cell supervisors	Periodic, coordinated.
Adapter	Protocol bridges, vendor drivers	With native stack.
Device	Actuators, sensors, end effectors	Rare, high-consequence.

Because roles evolve independently, OpenROC requires **backward-compatible negotiation** rather than lockstep upgrades.

10.3 Version Negotiation

`roc` is `MAJOR.MINOR`. Compatibility rules:

- Same major: interoperable. New optional fields, verbs, and states are ignored by older peers.
- Newer minor: an older peer may still communicate as long as it does not need the new features.
- Different major: not interoperable; the connection is refused with a clear reason.

Peers exchange versions during discovery:

```
{
  "type": "query",
  "verb": "capabilities",
  "payload": { "roc": "1.0", "encodings": ["json", "bin"], "planes": ["action", "
     actuation"] }
}
```

```
{
  "type": "response",
  "verb": "capabilities",
  "payload": {
    "roc": "1.2",
    "min_major": 1,
    "encodings": ["json", "bin"],
    "features": ["scheduled_setpoints", "impedance"],
    "profile": "actuation"
  }
}
```

The effective feature set is the intersection of both peers' features. A peer MUST NOT use a feature the other did not advertise.

10.4 Capability Handshake

After version negotiation, peers exchange endpoint descriptors (see **OpenROC Adapters**) and establish:

- accepted encodings per endpoint;
- numeric skill and actuator ids for BIN;
- QoS guarantees and dedup windows;
- sync groups and time base;
- accepted signature algorithms and key ids.

This handshake is repeated on reconnect and after any update that changes capabilities.

10.5 Update Artifacts

Update artifacts are content-addressed and signed:

```
{
  "artifact": {
    "name": "controller-ref",
    "version": "2.4.1",
    "target_roc": "1.x",
    "digest": "sha256:9f2c...",
    "signature": { "alg": "ES256", "key_id": "vendor-acme#release" },
    "size": 7340032,
    "url": "https://updates.example.com/controller-ref/2.4.1"
  }
}
```



Figure 11: Update Flow

A device accepts an artifact only if its digest matches and its signature chains to a trusted release key.

10.6 Update Flow

1. **Announce.** An update coordinator advertises an artifact for a target role and version range.
2. **Preflight.** The target verifies compatibility with its `roc` major, hardware revision, and available storage.
3. **Download.** The artifact is fetched, resumed if interrupted, and verified by digest.
4. **Stage.** The artifact is written to an inactive slot; the running system is untouched.
5. **Verify.** Signature and, optionally, a self-test of the staged image.
6. **Activate.** The target switches to the new image at a safe point, coordinated with the cell so no action is mid-execution.
7. **Confirm.** The target reports a new capability descriptor; the coordinator verifies the effective feature set.
8. **Commit or Roll Back.** If health checks pass, the update is committed; otherwise the target restarts into the previous slot.

The `roc` protocol treats an update as a **state transition** of the endpoint: during update the endpoint reports `state: updating`, rejects `operational` commands, and continues to honor `emergency` and `safety` traffic.

10.7 Atomicity and Rollback

Updates are atomic with respect to command execution:

- Activation happens only when the target is idle or explicitly drained.
- The previous version remains available in an inactive slot.
- A failed health check triggers automatic rollback, restoring the prior capability descriptor and version.
- Rollback is reported as a distinct audit event with the reason.

10.8 Staged Rollout

Coordinators roll out in stages to limit blast radius:

Stage	Population	Gate
Canary	1 endpoint	Health + conformance.
Pilot	A single cell	Task success rate, no safety events.
Broad	Site-wide	Error budget maintained.
Fleet	All sites	Manual promotion.

Each stage has explicit promotion and abort criteria. Aborting a stage triggers rollback across the staged population.

10.9 Coordinated Updates

When multiple roles must update together (for example, a controller and its adapter), the coordinator uses a **transaction**:

1. Negotiate a target feature set acceptable to all participants.
2. Stage all artifacts before activating any.
3. Activate in dependency order (devices, adapters, controllers, planners).
4. Abort and roll back all participants if any activation fails.

Because peers negotiate features rather than assume them, a partial or delayed update does not corrupt the system; it simply runs with the intersection of capabilities.

10.10 Monitoring After Update

The capability descriptor is the contract after an update. Coordinators monitor:

- reconnect and re-handshake success rate;
- command admission rate and rejection reason codes;

- safety events and watchdog expiries;
- declared vs. measured timing properties.

  A regression in any metric beyond its error budget triggers automatic rollback.

10.11 Security of Updates

Update security uses the same trust model as command security (see **Security & Safety**):

- artifacts are signed by an offline release key;
- devices pin the trust root and reject unknown signers;
- downloads use authenticated, encrypted transports;
- anti-rollback counters prevent downgrade to a vulnerable version unless explicitly authorized.

10.12 Conformance Notes

A deployment that supports updates must implement version negotiation, capability re-handshake, signed artifact verification, atomic activation with rollback, and the `updating` endpoint state. The effective feature set after an update must be re-established through negotiation and reported in the endpoint descriptor before `operational` commands resume.